
文档项目模板

发行版本 1.0

AWA2253

2025 年 02 月 12 日

1	RV 验证固件	3
1.1	M33、DSP 均以默认频率运行，以 DPLL1 可配置频率为主，默认频率为 M33-192M, RV-533M, DSP-400M	3
1.2	将 HS-PSRAM 从 DPLL3 切换到 DPLL1 中，频率根据当前条件尽量配高频，默认运行	4
1.3	将 RV 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 RV 使用	5
1.4	支持 RV 跑 memtester，跑在 HS-PSRAM 上	7
1.5	支持查看 RV 占用率，确保 RV 运行 memtester 正常	10
1.6	将 RV、M33 内部 LDO 供电电压修改至 1.08V，DSP 内部供电保持 1.175V 不变	10
2	验证	13
2.1	1、M33、DSP 均以默认频率运行，以 DPLL1 可配置频率为主，默认频率为 M33-192M, RV-533M, DSP-400M;	13
2.2	2、将 HS-PSRAM 从 DPLL3 切换到 DPLL1 中，频率根据当前条件尽量配高频，默认运行;	13
2.3	3.1、将 RV 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 RV 使用;	14
2.4	3.2、将 M33 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 M33 使用;	14
2.5	4、支持 RV 跑 memtester，跑在 HS-PSRAM 上;	14
2.6	5、支持查看 RV 占用率，确保 RV 运行 memtester 正常;	16
3	M33 验证固件	19
3.1	将 M33 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 M33 使用	19
4	查看 cpu 频率	21
4.1	M33 和 C906	21
4.2	HPSRAM	23
5	MyST Markdown 语法入门	25
5.1	提示框	25
5.2	标题	26
5.3	有序列表	26
5.4	无序列表	26
5.5	文本换行	28
5.5.1	无效换行	28
5.5.2	段内换行	28
5.5.3	新段落	28
5.6	字体样式	28
5.7	单行块引用	29
5.8	多行块引用	29

5.9	代码块	30
5.10	文件中的代码	30
5.11	数学公式	31
5.12	脚注	31
5.13	嵌入图像	31
5.14	超链接	33
5.15	表格	33
6	Indices and tables	35

pccli 命令可以执行他核的命令，用法：

rpccli [armldsplrv] commandname [arg0 ...]

1.1 M33、DSP 均以默认频率运行，以 DPLL1 可配置频率为主，默认频率为 M33-192M，RV-533M，DSP-400M

提供 cpu 频率查看 cmd 接口，补丁内容如下：

源文件：r128-dev/lichee/rtos/arch/risc-v/sun20iw2p1/cpufreq.c

```
#include <stdio.h>
#include <console.h>
#include <FreeRTOS.h>int cmd_get_cpu_freq(void)
{
    uint32_t cpu_freq = 0;
    int ret = 0;

    /*get cpu frequency */
    ret = get_cpu_freq(&cpu_freq);
    if (ret) {
        printf("get_cpu_freq failed, ret: %d\n", ret);
        return -1;
    }
    printf("get_cpu_freq ok, frequency: %d Hz\n", cpu_freq);
    return 0;
}
FINSH_FUNCTION_EXPORT_CMD(cmd_get_cpu_freq, get_cpu_freq, get current cpu frequency);
```

1.2 将 HS-PSRAM 从 DPLL3 切换到 DPLL1 中，频率根据当前条件尽量配高频，默认运行

更改 HS-PSRAM 频率的相关源文件：

r128-dev/lichee/brandy-2.0/spl/drivers/hpsram/hpsram.c

相关函数：

hpsram_init：开机默认初始化 hpsram 参数，其中包括了 HPSRAM 的 PLL 选择和频率设置。

hpsram_clk_init：设置 PSRAM 的 PLL 来源，分频系数等。

```
// sr32(REG_ADDR, num_bit, len, val);表示对寄存器地址REG_ADDR的num_bit:num_bit-
→1的位数赋值为val
static __inline void hpsram_clk_init(__dram_para_t *para)
{
    ...
    if (((para->dram_tpr7>>0) & 0x1) == 0) {
        printf("DPLL1 is chosen\n");
        sr32(DEV_CLK_CTRL, 21, 1, 0); //choose source DPLL1
        sr32(DPLL1_OUT_CTRL, 15, 1, 1); //enable DPLL1 clock output
        sr32(DPLL3_OUT_CTRL, 15, 1, 0); //disable DPLL3 clock output
    } else {
        printf("DPLL3 is chosen\n");
        sr32(DEV_CLK_CTRL, 21, 1, 1); //choose source DPLL3
        // CCMU_AON DPLL LS clk
        sr32(DPLL1_OUT_CTRL, 15, 1, 0); //disable DPLL1 clock output
        sr32(DPLL3_OUT_CTRL, 15, 1, 1); //enable DPLL3 clock output
    }
    ...
}
```

从源代码中可以知道，HSPSRAM 选择时钟来源可以选择 DPLL1 和 DPLL3，通过 para->dram_tpr7 来决定，如果为 0 则选择 DPLL1，如果为 1 则选择 DPLL3。而 dram_tpr7 的值可以通过 sys_config.fex 来设置，如下：

```
//sys_config.fex
;for compatibility, provide the 'dram_para' configuration. otherwise update_boot0_
→program will exit without writing boot0 header
; hpsram init if (dram_clk != 0); lpsram init if ( dram_no_lpsram == 0)
[dram_para]
...
dram_tpr7      = 0x0
...
```

另外，DPLL 的时钟分频系数通过可以由以下代码设置：

```
//choose DIV for ddr clk
//CK1_HSPSRAM_DIV 0:M=3; 1:M=2.5; 2:M=2; 3:M=1
sr32(DPLL1_OUT_CTRL, 12, 2, 3);
//CK3_HSPSRAM_DIV 00:M=3;01:M=2.5;10:M=2;11:M=1
sr32(DPLL3_OUT_CTRL, 12, 2, 3);
```

要求中尽量将 DPLL1 设置高频，因此需要将分频系数设置为 M=1。

1.3 将 RV 挂载到 DPLL3 上, DPLL3 的时钟源尽量只给 RV 使用

RV 核的 DPLL 配置相关源文件:

r128-dev/lichee/rtos/arch/arm/armv8m/sun20iw2p1/sun20i.c

相关函数:

sun20i_boot_c906: 启动 c906 时对其进行配置初始化, 包括 DPLL 配置。

相关代码段:

```
#define RISC_V_PLL_CLK_SOURCE RISC_V_DPLL3_CLK
...
#if (RISC_V_PLL_CLK_SOURCE == RISC_V_DPLL3_CLK)
    clk_ck3_c906 = hal_clock_get(HAL_SUNXI_AON_CCUC, CLK_CK3_C906);
    ret = hal_clock_enable(clk_ck3_c906);
    if (HAL_CLK_STATUS_OK != ret) {
        ret = -1;
        goto exit_with_put_dpll3_clk;
    }

    ret = hal_clk_set_rate(clk_ck3_c906, RISC_V_PLL_CLK_FREQ);
    if (HAL_CLK_STATUS_OK != ret) {
        ret = -1;
        goto exit_with_put_dpll3_clk;
    }
}
#endif

#if (RISC_V_PLL_CLK_SOURCE == RISC_V_DPLL1_CLK)
    //set clk_ck1_c906 clk to 480M
    //sr32(CCMU_AON_BASE+0xa4, 4, 3, 0x1);
    //set clk_ck1_c906 clk to 640M
    //sr32(CCMU_AON_BASE+0xa4, 4, 3, 0x2);
    ret = hal_clk_set_rate(clk_ck1_c906, RISC_V_PLL_CLK_FREQ);
    if (HAL_CLK_STATUS_OK != ret) {
        ret = -1;
        goto err2;
    }
}
#endif
```

根据以上代码内容可知, C906 选择 DPLL 由 RISC_V_PLL_CLK_SOURCE 决定, 只需要定义 RISC_V_PLL_CLK_SOURCE 为 RISC_V_DPLL3_CLK 即可完成将 RV 挂载到 DPLL3 上。

执行 p 0x4004c4e0 查看寄存器的值为 0x00062191, 因此, 以上设备有 DSP 和 RV 挂载在 DPLL3 上, 其他设备挂载在 DPLL1 上, 需要将 DSP 重新挂载在 DPLL1 上。

Offset:0x0E0			Register Name: DEV_CLK_CTRL
Bit	Read/Write	Default/Hex	Description
31:22	/	/	/
21 0	R/W	0x0	CKPLL_HSPSRAM_SEL select HSPSRAM clock source from DPLL1 and DPLL3 0: CK1_HSPSRAM 1: CK3_HSPSRAM
20 0	R/W	0x0	CKPLL_LSPSRAM_SEL Select LSPSRAM clock source from DPLL1 and DPLL3 0: CK1_LSPSRAM 1: CK3_LSPSRAM
19 0	R/W	0x0	CKPLL_M33_SEL Select M33 CPU clock source from DPLL1 and DPLL3 0: CK1_M33 1: CK3_M33
18 1	R/W	0x0	CKPLL_HIFI5_SEL select HIFI5 CPU clock source from DPLL1 and DPLL3 0: CK1_HIFI5 1: CK3_HIFI5
17 1	R/W	0x0	CKPLL_C906_SEL select C906 DSP clock source from DPLL1 and DPLL3 0: CK1_906 1: CK3_906
16 0	R/W	0x0	CKPLL_DEV_SEL select device clock source from DPLL1 and DPLL3 0: CK1_DEV 1: CK3_DEV

添加宏定义选择 DSP 挂载到 DPLL1 上，补丁内容如下：

```
diff --git a/arch/arm/armv8m/sun20iw2p1/sun20i.c b/arch/arm/armv8m/sun20iw2p1/sun20i.c
index 00999ef33..41d94f419 100755
--- a/arch/arm/armv8m/sun20iw2p1/sun20i.c
+++ b/arch/arm/armv8m/sun20iw2p1/sun20i.c
@@ -951,6 +951,10 @@ err1:
 }
 #endif

+#define DSP_DPLL1_CLK 1
+#define DSP_DPLL3_CLK 3
+#define DSP_PLL_CLK_SOURCE DSP_DPLL1_CLK
+
 #if defined(CONFIG_ARCH_ARMV8M_DEFAULT_BOOT_DSP) || defined(CONFIG_COMMAND_BOOT_DSP)
 ↪ \
     || defined(CONFIG_PM_SUBSYS_DSP_SUPPORT)
 #define DSP_CORE_CLOCK_FREQ CONFIG_DSP_CLOCK_FREQ
@@ -999,9 +1003,25 @@ int __sun20i_boot_dsp_with_start_addr(uint32_t dsp_start_addr)
     goto err2;
```

(续下页)

(接上页)

```

    }

-    //set clk_ck1_hifi5 clk to 384M
-    /* sr32(CCMU_AON_BASE+0xa4, 8, 3, 0x2); */
+#if (DSP_PLL_CLK_SOURCE == DSP_DPLL1_CLK)
+#define DSP_CORE_CLOCK_FREQ 384000000
+    // set clk_ck1_hifi5 clk to 384M
+    // sr32(CCMU_AON_BASE+0xa4, 8, 3, 0x2);
+    ret = hal_clk_set_rate(clk_ck1_hifi5, DSP_CORE_CLOCK_FREQ);
+    if (HAL_CLK_STATUS_OK != ret) {
+        ret = -1;
+        goto err1;
+    }

+    /*set clk_ck_hifi5 source to clk_ck1_hifi5*/
+    // sr32(CCMU_AON_BASE+0xe0, 18, 1, 0x0);
+    clk_ck_hifi5 = hal_clock_get(HAL_SUNXI_AON_CCU, CLK_CKPLL_HIFI5_SEL);
+    ret = hal_clk_set_parent(clk_ck_hifi5, clk_ck1_hifi5);
+    if (HAL_CLK_STATUS_OK != ret) {
+        ret = -1;
+        goto err3;
+    }
+    }
+}
+
+//set clk_ck3_hifi5 clk to 400M
+//sr32(CCMU_AON_BASE+0xa8, 8, 3, 0x3);
+ret = hal_clk_set_rate(clk_ck3_hifi5, DSP_CORE_CLOCK_FREQ);
@@ -1018,6 +1038,7 @@ int __sun20i_boot_dsp_with_start_addr(uint32_t dsp_start_addr)
    ret = -1;
    goto err3;
}
}
+}
+
+//set clk_ck_hifi5_div source to clk_ck_hifi5
+//sr32(CCMU_BASE+0x068, 4, 2, 0x2);

```

重新执行 `p 0x4004c4e0` 查看寄存器的值为 `0x00022191`，可以知道 DSP 已成功挂载到 DPLL1 上了。

1.4 支持 RV 跑 memtester，跑在 HS-PSRAM 上

支持 RV 跑 memtester

用法: `memtester -s {内存大小} [-m testmask] [-t 循环次数]`

比如: `memtester -s 8k -t 2`

```

"Usage: %s -s <size>[B|K|M|G] [-m testmask] [-t times]\n"
"\n"
"[testmask]: specify tests which we want run.\n"
"bit0  Compare XOR\n"
"bit1  Compare SUB\n"
"bit2  Compare MUL\n"
"bit3  Compare DIV\n"
"bit4  Compare OR\n"
"bit5  Compare AND\n"
"bit6  Sequential Increment\n"

```

(续下页)

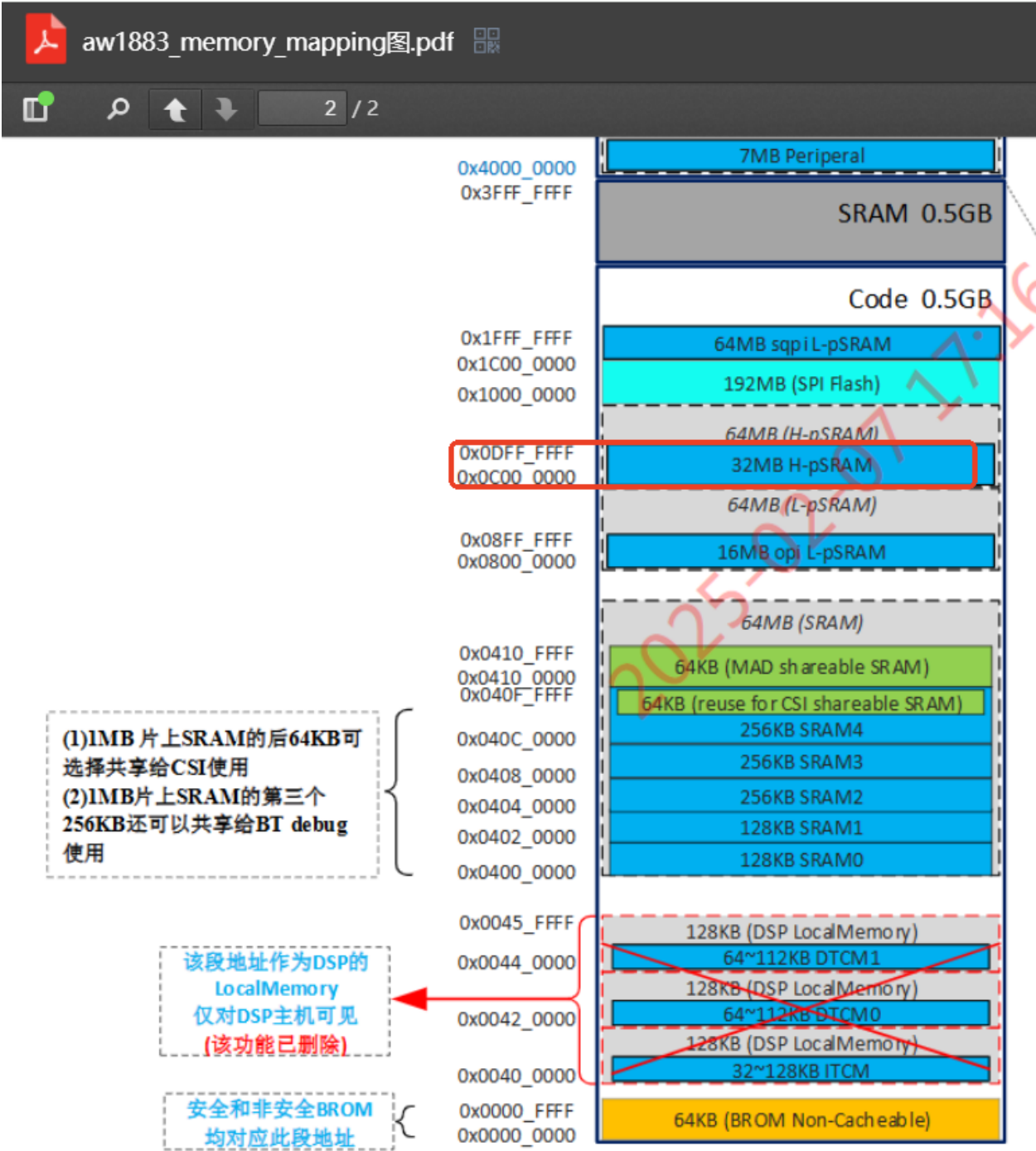
(接上页)

```
"bit7  Solid Bits\n"
"bit8  Block Sequential\n"
"bit9  Checkerboard\n"
"bit10 Bit Spread\n"
"bit11 Bit Flip\n"
"bit12 Walking Ones\n"
"bit13 Walking Zeroes\n"
"bit14 8-bit Writes(ifdef TEST_NARROW_WRITE)\n"
"bit15 16-bit Writes(ifdef TEST_NARROW_WRITE)\n"
"\n"
"eg:\n"
"    memtester -s 10M -m 0xc -t 1\n"
"    means run Compare DIV&MUL tests only to test 10M memory loop 1 time.\n",
```

RV 跑在 HS-PSRAM 上

RV 核有可能跑在 HS-PSRAM 或者 SRAM 上, 可以通过加载地址确认。

根据 memory mapping 图, HS-PSRAM 起始地址为 0x0C 开头, SRAM 起始地址为 0x04 开头, 因此通过打印 RV 核的起始地址即可判断 RV 核运行在哪个位置上。



RV 核启动地址相关代码：
相关代码源文件：r128-dev/lichee/rtos/arch/arm/armv8m/sun20iw2p1/sun20i.c
相关源代码：

```
int sun20i_boot_c906(void)
{
...
    #if CONFIG_ARCH_RISCV_START_ADDRESS
```

(续下页)

(接上页)

```

        put_wvalue(RISCV_CFG_BASE + 0x004, CONFIG_ARCH_RISCV_START_ADDRESS);
    #else
        put_wvalue(RISCV_CFG_BASE + 0x004, rtos_spare_head.reserved[0]);
    #endif
    ...
}

```

可通过插入打印 CONFIG_ARCH_RISCV_START_ADDRESS 的值直接确定 RV 核的起始地址，从而确定运行位置。

也可以直接根据 boot0 启动时输出的 log 来确定：

```

[294]loadparts:arm-hpsram@:rv-hpsram@:dsp-hpsram@:config@0xc000000
[300]load_start_sector=41008
[303]load arm-hpsram to 0xc004000 from 20996224, size 1186320
[308]load_start_sector=41008, load_sectors=2317, loadaddr=0x0c004000,first_part_
↪size=0x00000180
[419]load_start_sector=43408
[422]load rv-hpsram to 0xc200000 from 22225024, size 4031544
[428]load_start_sector=43408, load_sectors=7874, loadaddr=0x0c200000,first_part_
↪size=0x00000180
[783]load_start_sector=51288
[786]load dsp-hpsram to 0xd800000 from 26259584, size 503924
[792]load_start_sector=51288, load_sectors=984, loadaddr=0x0d800000,first_part_
↪size=0x00000180
[844]load_start_sector=55288

```

通过打印可知，RV 核的起始地址为 0xc200000，ARM 核的起始地址为 0x0c00400，故而可以 RV 和 ARM 都已经运行在 HS-PSRAM 上。

1.5 支持查看 RV 占用率，确保 RV 运行 memtester 正常

1.6 将 RV、M33 内部 LDO 供电电压修改至 1.08V，DSP 内部供电保持 1.175V 不变

相关源文件：r128-dev/lichee/rtos/arch/risc-v/sun20iw2p1/cpufreq.c

相关接口：

set_cpu_voltage：设置当前 cpu LDO 电压

get_cpu_voltage：获取当前 cpu LDO 电压

在上面接口基础上添加 cmd 接口，补丁内容如下：

```

diff --git a/arch/risc-v/sun20iw2p1/cpufreq.c b/arch/risc-v/sun20iw2p1/cpufreq.c
index e5329c00f..1a2a58b32 100644
--- a/arch/risc-v/sun20iw2p1/cpufreq.c
+++ b/arch/risc-v/sun20iw2p1/cpufreq.c
@@ -32,6 +32,9 @@

#include "cpufreq.h"
#include <hal_clk.h>
+#include <stdio.h>

```

(续下页)

(接上页)

```

#include <console.h>
#include <FreeRTOS.h>
#ifdef CONFIG_DRIVERS_EFUSE
#include <sunxi_hal_efuse.h>
#endif
@@ -156,6 +159,42 @@ int get_cpu_voltage(uint32_t *cpu_voltage)
    return 0;
}

+static int cmd_set_cpu_voltage(int argc, const char **argv)
+{
+    uint32_t vol = 0;
+    int ret = 0;
+    if (argv[1] == NULL) {
+        printf("please input cpu voltage (mV)\n");
+        return -1;
+    }
+
+    vol = strtol(argv[1], NULL, 0);
+    ret = set_cpu_voltage(vol);
+    if (ret) {
+        printf("set_cpu_voltage failed, ret: %d\n", ret);
+        return -1;
+    }
+    printf("set_cpu_voltage ok, voltage: %d mV\n", vol);
+    return 0;
+}
+FINSH_FUNCTION_EXPORT_CMD(cmd_set_cpu_voltage, set_cpu_voltage, set current cpu_
+↪voltage);
+
+static int cmd_get_cpu_voltage(void)
+{
+    uint32_t cpu_voltage = 0;
+    int ret = 0;
+
+    /*get cpu voltage */
+    ret = get_cpu_voltage(&cpu_voltage);
+    if (ret) {
+        printf("get_cpu_voltage failed, ret: %d\n", ret);
+        return -1;
+    }
+    printf("get_cpu_voltage ok, voltage: %d mV\n", cpu_voltage);
+    return 0;
+}
+FINSH_FUNCTION_EXPORT_CMD(cmd_get_cpu_voltage, get_cpu_voltage, get current cpu_
+↪voltage);
+
+static int set_first_div_freq(uint32_t target_freq, hal_clk_t next_clk)
+{
+    int ret = 0;
+}
@@ -339,3 +378,20 @@ int cpufreq_vf_init(void)
    return 0;
}

```

测试:

设置 LDO 电压为 1.08V:

```
c906>set_cpu_voltage 1080
set_cpu_voltage ok, voltage: 1080 mV

c906>get_cpu_voltage
get_cpu_voltage ok, voltage: 1100 mV
```

设置 LDO 电压 1.08V 成功, 但是最后读取出来的 LDO 电压为 1.1V。

原因:

根据宏定义 `#define VOLTAGE_STEP 25` 以及 `set_cpu_voltage` 函数的具体实现:

```
ldo_vol_field = ldo_cal + ((int)vol - LDO_CAL_BASE) / VOLTAGE_STEP;
```

可以知道步进电压为 25mV, 也就是设置的电压必须为 25mV 的整数倍, 因此最接近 1.08V 的电压为 1.075V, 可以尝试设置 LDO 电压值为 1075。

```
c906>set_cpu_voltage 1075
set_cpu_voltage ok, voltage: 1075 mV

c906>get_cpu_voltage
get_cpu_voltage ok, voltage: 1075 mV
```

可以看到成功设置为 LDO 电压为 1.075V。

2.1 1、M33、DSP 均以默认频率运行，以 DPLL1 可配置频率为主，默认频率为 M33-192M，RV-533M，DSP-400M；

M33 固件下，重启查看 log：

```
M33 CPU Clock Freq: 192 MHz
C906 CPU Freq: 480 MHz
DSP core clock frequency: 384000000Hz
```

RV 固件下，重启查看 log：

```
M33 CPU Clock Freq: 192 MHz
C906 CPU Freq: 533 MHz
DSP core clock frequency: 384000000Hz
```

2.2 2、将 HS-PSRAM 从 DPLL3 切换到 DPLL1 中，频率根据当前条件尽量配高频，默认运行；

在 RV 或者 M33 固件下，执行以下命令：

```
c906>p 0x4004c4e0 //读取system clock control寄存器
reg_addr[0x4004c4e0]=0x00022191 //bit21为0，说明HS-PSRAM时钟源为DPLL1
```

21	R/W	0x0	CKPLL_HSPSRAM_SEL select HSPRAM clock source from DPLL1 and DPLL3 0: CK1_HSPSRAM 1: CK3_HSPSRAM
----	-----	-----	--

2.3 3.1、将 RV 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 RV 使用；

在 RV 固件下，执行以下命令：

```
c906>p 0x4004c4e0 //读取system clock control寄存器
reg_addr[0x4004c4e0]=0x00022191 //bit17为1，说明RV时钟源为DPLL3
```

17	R/W	0x0	CKPLL_C906_SEL select C906 DSP clock source from DPLL1 and DPLL3 0: CK1_906 1: CK3_906
----	-----	-----	---

bit18~bit21 为 0，表示 HSPSRAM、LSPSRAM、M33、DSP 的时钟源都是 DPLL1，DPLL3 的时钟源只给 RV 使用。

2.4 3.2、将 M33 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 M33 使用；

在 M33 固件下，执行以下命令：

```
c906>p 0x4004c4e0//读取system clock control寄存器
reg_addr[0x4004c4e0]=0x00082191//bit19为1，说明M33时钟源为DPLL3
```

19	R/W	0x0	CKPLL_M33_SEL Select M33 CPU clock source from DPLL1 and DPLL3 0: CK1_M33 1: CK3_M33
----	-----	-----	---

bit18、bit20 和 bit21 为 0，表示 HSPSRAM、LSPSRAM、RV、DSP 的时钟源都是 DPLL1，DPLL3 的时钟源只给 M33 使用。

2.5 4、支持 RV 跑 memtester，跑在 HS-PSRAM 上；

在 RV 或者 M33 固件下，执行以下命令：

```
c906>memtester -s 8k -t 1
memtester version 4.3.0 (64-bit)
Copyright (C) 2001-2012 Charles Cazabon.
Licensed under the GNU General Public License version 2 (only).

sysconf(_SC_PAGE_SIZE) not supported; using pagesize of 8192
memsuffix k
want 0MB (8192 bytes)
got 0MB (8192 bytes) in 0xc643d20Loop 1/1:
<=====Prepare=====>
  Stuck Address      : ok
  Random Value       : ok
<=====Start test=====>
  Compare XOR        : ok
```

(续下页)

(接上页)

```

Compare SUB      : ok
Compare MUL      : ok
Compare DIV      : ok
Compare OR       : ok
Compare AND      : ok
Sequential Increment: ok
Solid Bits       : ok
Block Sequential : ok
Checkerboard     : ok
Bit Spread       : ok
Bit Flip         : ok
Walking Ones     : ok
Walking Zeroes   : ok

Done.

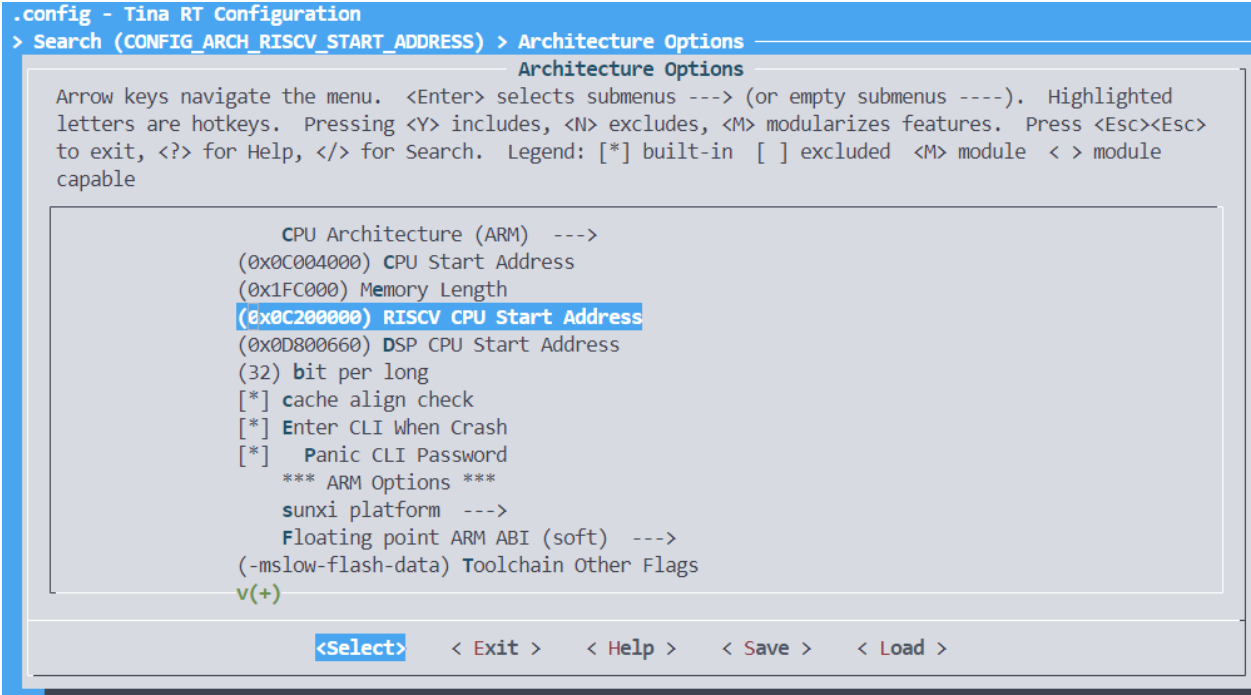
c906>rpccli arm memtester -s 8k -t 1
memtester version 4.3.0 (32-bit)
Copyright (C) 2001-2012 Charles Cazabon.
Licensed under the GNU General Public License version 2 (only).

sysconf(_SC_PAGE_SIZE) not supported; using pagesize of 8192
memsuffix k
want OMB (8192 bytes)
got OMB (8192 bytes) in 0xa5a5a5a50c13c220Loop 1/1:
<=====Prepare=====>
  Stuck Address      : ok
  Random Value       : ok
<=====Start test=====>
  Compare XOR        : ok
  Compare SUB        : ok
  Compare MUL        : ok
  Compare DIV        : ok
  Compare OR         : ok
  Compare AND        : ok
  Sequential Increment: ok
  Solid Bits         : ok
  Block Sequential   : ok
  Checkerboard       : ok
  Bit Spread         : ok
  Bit Flip           : ok
  Walking Ones       : ok
  Walking Zeroes     : ok

Done.

```

查看 RV 核和 M33 核的启动地址:



地址为 0xC 开头，表明 RV 和 M33 都运行在 RPSRAM 中。

2.6 5、支持查看 RV 占用率，确保 RV 运行 memtester 正常；

RV 或 M33 固件下，直接运行 run-time-stats

```
c906>run-time-stats
```

Task	Abs Time	% Time

CLI	616	<1%
IDLE	516619	99%
adbd-input	0	<1%
tcpip	0	<1%
nand_rcd	0	<1%
nftld	0	<1%
amp-admin	0	<1%
Tmr Svc	0	<1%
amp-ser1	0	<1%
amp-ser2	0	<1%
amp-ser3	0	<1%
amp-ser4	0	<1%
pm_suspend	0	<1%
hub-main-thread	0	<1%
amp-send-task	0	<1%
amp-recv-task	0	<1%
udc_vbus_det	0	<1%
adbd-output	1	<1%
AudioMT2	0	<1%
cpu_zone	0	<1%
disp2	258	<1%
amp-ser0	0	<1%

6、将 RV、M33 内部 LDO 供电电压修改至 1.08V，DSP 内部 LDO 供电保持 1.175V 不变；

RV 固件下：

```
c906>set_cpu_voltage 1075 //设置RV内部LDO供电电压
set_cpu_voltage ok, voltage: 1075 mV

c906>get_cpu_voltage //读取RV内部LDO供电电压
get_cpu_voltage ok, voltage: 1075 mV
```

M33 固件下：

```
c906>rpccli arm set_cpu_voltage 1075 //设置M33内部LDO供电电压
set_cpu_voltage ok, voltage: 1075 mV

c906>rpccli arm get_cpu_voltage //读取M33内部LDO供电电压
get_cpu_voltage ok, voltage: 1075 mV
```


3.1 将 M33 挂载到 DPLL3 上，DPLL3 的时钟源尽量只给 M33 使用

相关源代码文件：r128-dev/lichee/rtos/arch/arm/armv8m/sun20iw2p1/sun20i.c

相关函数：static int ar200a_clk_set(u32 freq)

相关代码补丁：

```
diff --git a/arch/arm/armv8m/sun20iw2p1/sun20i.c b/arch/arm/armv8m/sun20iw2p1/sun20i.c
index 1c7cb1abd..509f3b516 100755
--- a/arch/arm/armv8m/sun20iw2p1/sun20i.c
+++ b/arch/arm/armv8m/sun20iw2p1/sun20i.c
@@ -173,12 +173,16 @@ err:
     * we fixed ck1_m33 to 384M,
     * so freq only support 384M,192M,128M,96M,64M,48M,32M,24M
     */
+ #define M33_DPLL1_CLK 1
+ #define M33_DPLL3_CLK 3
+ #define M33_PLL_CLK_SOURCE M33_DPLL3_CLK
     static int ar200a_clk_set(u32 freq)
     {
         u32 rate;
-        hal_clk_t clk_ck1_m33, clk_ck_m33, clk_sys, clk_ar200a_f;
+        hal_clk_t clk_ck1_m33, clk_ck3_m33, clk_ck_m33, clk_sys, clk_ar200a_f;
+        hal_clk_status_t ret = 0;

+ #if (M33_PLL_CLK_SOURCE == M33_DPLL1_CLK)
+         /*set ck1_m33_clk*/
+         //sr32(CCMU_AON_BASE+0xa4, 3, 1, 1);
+         //0x4004c4a4: 0x80800008
@@ -207,12 +211,25 @@ static int ar200a_clk_set(u32 freq)
         ret = -1;
         goto err3;
     }
}
```

(续下页)

(接上页)

```
-
+   printf("M33: chose clock source to DPLL1\n");
+#else
+   printf("DPLL3\n");
+   /*enable ck3_m33_clk*/
+   sr32(CCMU_AON_BASE+0xa8, 3, 1, 1);
+   //0x4004c4a4: 0x80800008
+   /*fixed 384M*/
+   sr32(CCMU_AON_BASE+0xa8, 0, 3, 0x3);
+   //0x4004c4a4: 0x8080000b
+   /*set ck_m33_clk*/
+   sr32(CCMU_AON_BASE+0xe0, 19, 1, 1);
+   //0x4004c4e0: 0x00000101
+   printf("M33: chose clock source to DPLL3\n");
+#endif
+   /*set sys_clk*/
+   //sr32(CCMU_AON_BASE+0xe0, 8, 4, 0x7);
+   //0x4004c4e0: 0x00000701
+   clk_sys = hal_clock_get(HAL_SUNXI_AON_CCU, CLK_SYS);
-   ret = hal_clk_set_rate(clk_sys, freq);
+   ret = hal_clk_set_rate(clk_sys, freq); //freq=192MHz
+   if (HAL_CLK_STATUS_OK != ret) {
+       ret = -1;
+       goto err4;
+   }
+   ...
```


4.1 M33 和 C906

(1) 确定 HOSC 时钟源:

```
p 0x4004c484 // 查看bit0:2的值， 根据下面信息直接确定f_hosc的值
```

1.5.2 HOSC Type Register(Default Value:0x0000_0000)

Offset:0x0084			Register Name: HOSC_TYPE
Bit	Read/Write	Default/Hex	Description
31:3	/	/	/
2:0	R/W	0x0	HOSC_TYPE 0: System uses 26MHz HOSC 1: System uses 40MHz HOSC 2: System uses 24MHz HOSC 3: System uses 32MHz HOSC 4: System uses 24.576MHz HOSC Others: reserved <i>Note: This bit is set by application software and must be consistent with the real situation. A wrong configuration will cause the RCOSC calibration fail.</i>

(2) 确定 DPLL3 的时钟频率:

```
p 0x4004c494 // f_dp113=(f_hosc*N)/M, N由bit4:11决定， M由0:3决定
```

1.5.6 DPLL3 Control Register(Default Value:0x6000_0181)

Offset:0x0094			Register Name: DPLL3_CTRL
Bit	Read/Write	Default/Hex	Description
31	R/W	0x0	PLL_ENABLE Enable or disable the system PLL. 0: Disable the system PLL 1: Enable the system PLL Note: The PLL frequency $F_{output} = (F_{hosc} * N.f) / M$, the output clock must be in range of 960MHz. The default value is set for 960MHz with 40MHz OSC.
30:29	R/W	0x3	DPLL_DITHER_DISABLE
28:13	R/W	0x0	DPLL_FRAC <i>Note : $V_{FRAC} = f * 2^{16}$</i>
12	R/W	0x0	DPLL_FRAC_CTRL
11:4	R/W	0x18	DPLL_NDIV PLL N.
3:0	R/W	0x1	FACTOR_M PLL factor M (1 ~ 8). M = Factor M <i>Note: when the factor is set to 0, M will be set to 1</i>

(3) 确定 M33 的频率

3	R/W	0x0	CK3_M33_EN 1: enable clock output 0: disable clock output
2:0	R/W	0x1	CK3_M33_DIV DPLL3 output factor M. 000: M=8; 001: M=7; 010: M=6; 011: M=5; 1XX: M=4; <i>The clock frequency $F_{ck3_m33} = F_{dpll3} / M$, F_{dpll3} is 960MHz ~ 1920MHz.</i>

```
c906>p 0x4004c4a8 // 查看bit0:2的值，直接确定分频系数M，那么f_m33=f_dp113/M
```

(4) 确定 C906 的频率

7	R/W	0x0	CK3_C906_EN 1: enable clock output 0: disable clock output
6:4	R/W	0x0	CK3_C906_DIV DPLL3 output factor M. 000: M=7; 001: M=4; 010: M=3; 011: M=2.5; 1XX: M=2; <i>The clock frequency $F_{ck3_906} = F_{dpll3} / M$, F_{dpll3} is 960MHz ~ 1920MHz.</i>

```
c906>p 0x4004c4a8 //查看bit4:6的值，直接确定分频系数M，那么f_c906=f_dpll3/M
```

4.2 HPSRAM

15	R/W	0x0	CK1_HPSRAM_EN 1: enable clock output 0: disable clock output
14	/	/	/
13:12	R/W	0x0	CK1_HPSRAM_DIV DPLL1 output factor M. 00: M=3; 01: M=2.5; 10: M=2; 11: M=1. <i>The clock frequency $F_{ck1_hpsram} = F_{dpll1} / M$, F_{dpll1} is 1920MHz.</i>

```
p 0x4004c4a8 //查看分频系数M，直接计算f_hpsram=f_dpll1/M
```

MyST Markdown 语法入门

MyST Markdown, MyST (全称 Markedly Structured Text) 专为 Sphinx 项目使用而设计的 Markdown 风格。它是 CommonMark Markdown 和一些额外的语法片段的组合, 以支持 Sphinx 的功能, 因此您可以用纯 Markdown 编写 Sphinx 文档。

[MyST-Parser 官网语法教程](#)

5.1 提示框

警告提示框 (Admonition) 突出显示与页面叙述稍有不同的特定文本块, 例如注释或警告。

```
 :::{tip}  
给读者一些有用的提示吧!  
 :::  
  
 :::{note}  
这是一个笔记  
 :::  
  
 :::{caution}  
这是一个警告  
 :::  
  
 :::{error}  
这是一个错误  
 :::
```

显示效果

:::{tip} 给读者一些有用的提示吧! :::

:::{note} 这是一个笔记:::

:::{caution} 这是一个警告:::

:::{error} 这是一个错误:::

5.2 标题

为了兼容考虑, 用一个空格在 # 和标题之间进行分隔。

```
# 一级标题

## 二级标题

### 三级标题

#### 四级标题

##### 五级标题
```

5.3 有序列表

要创建有序列表, 请在每个列表项前添加数字并紧跟一个英文句点。
数字不必按数学顺序排列, 但是列表应当以数字 1 起始。

```
1. 条目 1
2. 条目 2
3. 条目 3
   - 缩进条目
   - 缩进条目
```

显示效果

1. 条目 1
 2. 条目 2
 3. 条目 3
 - 缩进条目
 - 缩进条目
-

5.4 无序列表

要创建无序列表, 请在每个列表项前面添加破折号 (-)、星号 (*) 或加号 (+)。缩进一个或多个列表项可创建嵌套列表。

```
- 条目 1
- 条目 2
- 条目 3
- 条目 4
  - 缩进条目
  - 缩进条目

* 条目 1
* 条目 2
* 条目 3
* 条目 4
  * 缩进条目
  * 缩进条目

+ 条目 1
+ 条目 2
+ 条目 3
+ 条目 4
  + 缩进条目
  + 缩进条目
```

显示效果

- 条目 1
- 条目 2
- 条目 3
- 条目 4
 - 缩进条目
 - 缩进条目
- 条目 1
- 条目 2
- 条目 3
- 条目 4
 - 缩进条目
 - 缩进条目
- 条目 1
- 条目 2
- 条目 3
- 条目 4
 - 缩进条目
 - 缩进条目

5.5 文本换行

Markdown 中在段落前不要用空格 (spaces) 或制表符 (tabs) 缩进段落。

5.5.1 无效换行

在一行结尾没有加空格时, 后一行内容会自动追加显示在前一行

```
This is the first line.  
And this is the second line.
```

显示效果

This is the first line. And this is the second line.

5.5.2 段内换行

要在没有段落的情况下换行, 使用**两个或多个空格**进行换行, 称为结尾空格 (trailing whitespace)

```
This is the first line.  
And this is the second line.
```

显示效果

This is the first line.

And this is the second line.

5.5.3 新段落

两行之间**间隔一行**, 即可新增段落

```
This is the first line.  
  
And this is the second line.
```

显示效果

This is the first line.

And this is the second line.

5.6 字体样式

要加粗文本, 在短语前后各添加**两个星号** * 或下划线 _

要斜体文本, 在短语前后各添加**一个星号** * 或下划线 _

要同时用粗体和斜体突出显示文本, 在短语前后各添加**三个星号** * 或下划线 _

删除线在 PDF 中不可用

斜体

斜体

****粗体****

__粗体__

*****粗体+斜体*****

____粗体+斜体____

$H_{sub}2^O$, and $4^{sup}th$ of July

显示效果

斜体

斜体

粗体

粗体

粗体 + 斜体

粗体 + 斜体

H_2O , and 4th of July

5.7 单行块引用

要创建块引用，请在段落前添加一个> 符号。

> 这是一个引用块

显示效果

这是一个引用块

5.8 多行块引用

块引用可以包含多个段落。为段落之间的空白行添加一个> 符号。

> 呼啸的秋风让人无限忧愁，进也忧愁，退也忧愁。
>
> 异域戍边的人，哪个不陷入悲愁中？真是愁白了头啊。
>
> 胡人之处多狂风，树木萧瑟干枯。

显示效果

呼啸的秋风让人无限忧愁，进也忧愁，退也忧愁。
异域戍边的人，哪个不陷入悲愁中？真是愁白了头啊。
胡人之处多狂风，树木萧瑟干枯。

5.9 代码块

在代码前和后的行上使用三个反引号`````或三个波浪号`~~~` 在代码块之前的反引号旁边指定一种语言，即可实现语法高亮

```
```python
import os
print("Hello World!")
```

~~~ json
{
    "firstName": "John",
    "lastName": "Smith",
    "age": 25
}
~~~
```

显示效果

```
import os
print("Hello World!")
```

```
{
    "firstName": "John",
    "lastName": "Smith",
    "age": 25
}
```

5.10 文件中的代码

可以使用 `literalinclude` 指令从文件中包含更长的代码：

注意：“和 `{literalinclude}` 之间不能有空格

```
```{literalinclude} quickstart.py
```
```

5.11 数学公式

`math` 角色和指令分别用于定义内联数学和块数学。

注意: “和 `{math}`” 之间不能有空格

```
```{math}
:label: mymath
(a + b)^2 = a^2 + 2ab + b^2

(a + b)^2 &= (a + b) (a + b) \\
 &= a^2 + 2ab + b^2
```
```

显示效果

$$(a + b)^2 = a^2 + 2ab + b^2$$

$$(a + b)^2 = (a + b)(a + b)$$

$$= a^2 + 2ab + b^2$$
(5.1)

5.12 脚注

脚注使用 `pandoc` 规范。它们的标签以 `^` 开头，然后可以是任何字母数字字符串（无空格），不区分大小写。

```
- This is a manually-numbered footnote reference.[^3]
- This is an auto-numbered footnote reference.[^myref]

[^myref]: This is an auto-numbered footnote definition.
[^3]: This is a manually-numbered footnote definition.
```

显示效果

- This is a manually-numbered footnote reference.³
- This is an auto-numbered footnote reference.¹

5.13 嵌入图像

要添加图像，请使用感叹号 (!)，然后在方括号增加替代文本，图片链接放在圆括号里，括号里的链接后可以增加一个可选的图片标题文本。

```
![这是图片](_static/philly-magic-garden.jpg "Magic Gardens")

```{image} _static/philly-magic-garden.jpg
:alt: philly magic garden
:class: bg-primary
```

(续下页)

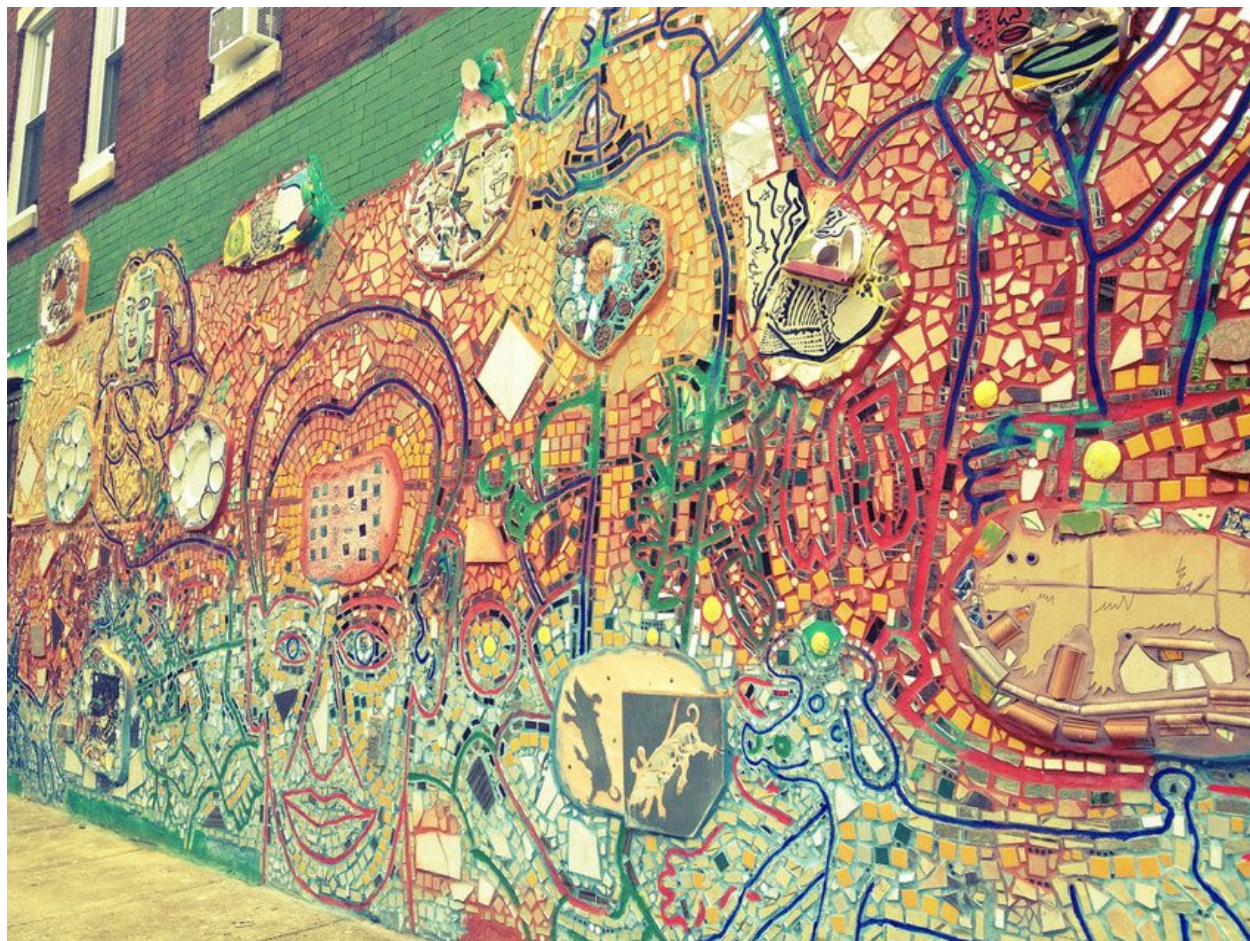
<sup>3</sup> This is a manually-numbered footnote definition.

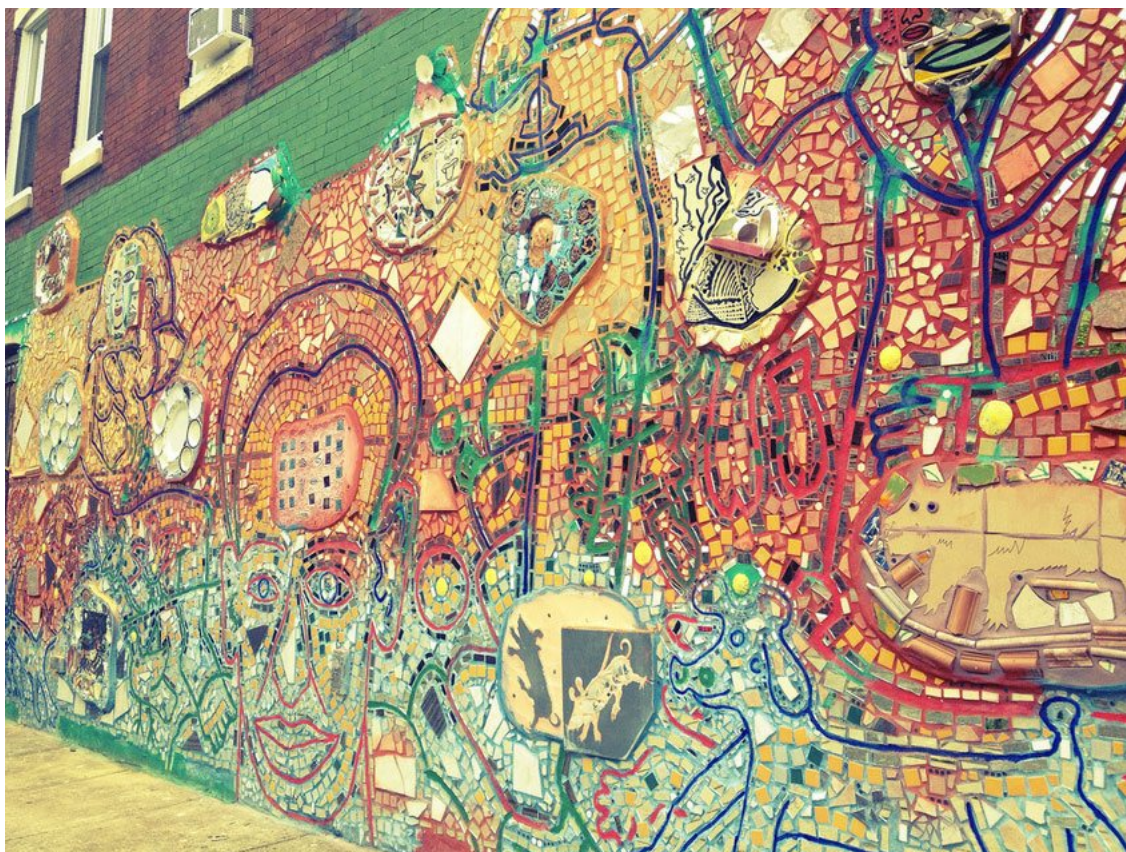
<sup>1</sup> This is an auto-numbered footnote definition.

(接上页)

```
:scale: 50 %
:align: center
```\n
```

显示效果





5.14 超链接

超链接 Markdown 语法代码: [超链接显示名] (超链接地址" 超链接 title")

这是一个链接 [Markdown 语法] (https://markdown.com.cn)

显示效果

这是一个链接 [Markdown 语法](https://markdown.com.cn)

5.15 表格

表格必须使用 MyST Markdown 格式, 否则 PDF 无法显示。

`list-table` 指令用于根据统一的两级项目符号列表中的数据创建表格。“统一”是指每个子列表（二级列表）必须包含相同数量的列表项。

```
:::{list-table} 表格标题
:widths: 15 10 30
:header-rows: 1
```

(续下页)

(接上页)

```
*   - **Treat**
    - **Quantity**
    - **Description**
*   - Albatross
    - 2.99
    - On a stick!
*   - Crunchy Frog
    - 1.49
    - If we took the bones out, it wouldn't be
      crunchy, now would it?
*   - Gannet Ripple
    - 1.99
    - On a stick!
:::
```

显示效果

```
:::{list-table} 表格标题:widths: 15 10 30 :header-rows: 1
```

- - **Treat**
 - **Quantity**
 - **Description**
 - - Albatross
 - 2.99
 - On a stick!
 - - Crunchy Frog
 - 1.49
 - If we took the bones out, it wouldn't be crunchy, now would it?
 - - Gannet Ripple
 - 1.99
 - On a stick! :::
-

CHAPTER 6

Indices and tables

- `genindex`
- `modindex`
- `search`